

```
1 import os
2 import pickle
3 import sys
4 import numpy as np
5 import model as nn
6 import copy
7 import scipy
8 import scipy.io
9 import argparse
10 from tqdm import tqdm
11 from utils import rotate_3d_coordinates
12 import matplotlib.pyplot as plt
13
14 def train(nnsgd, nnavg, R, Z, E_true, mb, hist,
15         augment):
16     rmses = []
17     for i in tqdm(range(1,100000)):
18
19         # learning rate schedule
20         if i > 0:      lr = 0.0000001
21         if i > 500:    lr = 0.00000025
22         if i > 2500:   lr = 0.0000005
23         if i > 12500:  lr = 0.000001
24         lr = lr*0.2
25
26         #sample minibatch indices
27         r = np.random.randint(0,len(R),[mb])
28         if augment:
29
30             R_aug = np.empty_like(R[r])
31
32             for j in range(mb):
33                 x_angle = np.random.uniform(-180, 180
34             )
35                 y_angle = np.random.uniform(-180, 180
36             )
37                 z_angle = np.random.uniform(-180, 180
```

```

35 )
36         #I realize that this now takes
        radians as argument, however, mathematically it
        shouldn't matter (even random dist for a sinusoidal
        fn)
37         R_aug[j] = rotate_3d_coordinates(R[r
        ][j], x_angle, y_angle, z_angle)
38         R_batch = R_aug
39     else:
40         R_batch = R[r]
41         E_pred = nnsgd.forward((R_batch, Z[r]))
42         loss = 0.5 * np.sum((E_pred - E_true[r]) ** 2
    ) # MSE loss
43
44         rmse = np.square(E_pred-E_true[r]).mean(axis=
    0)**.5
45         rmses.append(rmse)
46         nnsgd.backward((E_pred- E_true[r])) #TODO:
        add argument to backward
47         nnsgd.update(lr)
48         nnavg.average(nnsgd,(1/hist)/((1/hist)+i))
49         nnavg.nbiter = i
50
51         if i % 100 == 0:
52             print(f"RMSE: {sum(rmses[-100:])/100}
        kcal/mol")
53             pickle.dump(nnavg,open(f'nn-augment={
        augment}.pkl','wb'),pickle.HIGHEST_PROTOCOL)
54             np.save(f'training-rmses-augment={augment}.npy',
        np.array(rmses))
55             print("Done Training")
56
57
58 if __name__ == '__main__':
59     '''Setup'''
60     parser = argparse.ArgumentParser()
61     parser.add_argument('--seed', type=int, default=
    123, help='Random seed')

```

```

62     parser.add_argument('--augmentation', action='
store_true', help='whether to perform data
augmentations')
63     parser.add_argument('--train_fraction', type=
float, default=0.5, help='Fraction of training split
to use')
64     parser.add_argument('--mb', type=int, default=25
, help='Batch size')
65     parser.add_argument('--hist', type=float, default
=0.1, help='Fraction of the history to be remembered
for Exponential Moving Average')
66     args = parser.parse_args()
67
68     np.random.seed(args.seed)
69
70     '''Load data'''
71     if not os.path.exists('qm7.mat'): os.system('wget
http://www.quantum-machine.org/data/qm7.mat')
72     dataset = scipy.io.loadmat('qm7.mat')
73     split_idx = dataset['P'][1:].flatten() #leave
first split for testing
74     skip_every = int(1/args.train_fraction)
75     R = dataset['R'][split_idx][::skip_every]
76     Z = dataset['Z'][split_idx][::skip_every]
77     E_true = dataset['T'][0,split_idx][::skip_every]
78
79     '''Create neural network'''
80     #Define input and output layers
81     I, O = nn.Input((R, Z)), nn.Output(E_true)
82     nnsgd = nn.Sequential([
83         I,
84         nn.Linear(92, 400),
85         nn.Tanh(),
86         nn.Linear(400, 100),
87         nn.Tanh(),
88         nn.Linear(100, 1),
89         O
90     ])

```

```
91     nnavg = copy.deepcopy(nnsgd)
92
93     '''Train neural network'''
94     train(nnsgd, nnavg, R, Z, E_true, args.mb, args.
    hist, augment= True)
95
```

```
1 import numpy as np
2
3 class Module:
4     def update(self,lr): pass
5     def average(self,nn,a): pass
6     def backward(self,DY): pass
7     def forward(self,X): pass
8
9 class Sequential(Module):
10
11     def __init__(self,modules):
12         self.modules = modules
13
14     def forward(self,X):
15         '''Given input X, perform a full forward pass
16         through the MLP'''
17         #TODO: fill in this function
18         for m in self.modules:
19             X = m.forward(X)
20         return X
21
22     def backward(self,DY):
23         '''Perform a full backward pass through the
24         MLP.
25         DY is gradient of the loss w.r.t the
26         final output'''
27         #TODO: fill in this function
28         for m in reversed(self.modules):
29             DY = m.backward(DY)
30         return DY
31
32     def update(self,lr):
33         for m in self.modules:
34             X = m.update(lr)
35
36     def average(self,nn,a):
37         for m,n in zip(self.modules,nn.modules):
```

```

36         m.average(n,a)
37
38 class Input(Module):
39     def __init__(self, inp):
40         R, Z = inp
41         sample_in = np.concatenate([R, np.expand_dims
42 (Z, -1)], axis = -1)
43         self.nbout = sample_in.shape[-2] * sample_in.
44 shape[-1]
45
46     def forward(self,inp):
47         R, Z = inp
48         rz = np.concatenate([R, np.expand_dims(Z, -1
49 )], axis = -1)
50         return rz.reshape(rz.shape[0], -1)
51
52 class Output(Module):
53
54     def __init__(self,T):
55         self.tmean = T.mean()
56         self.tstd = T.std()
57         self.nbinp = 1
58
59     def forward(self,X):
60         # un-normalize the final prediction
61         self.X = X.flatten()
62         return self.X*self.tstd+self.tmean
63
64     def backward(self,DY):
65         #TODO: fill in this function
66         return DY * self.tstd
67
68 class Linear(Module):
69
70     def __init__(self,m,n):
71         self.lr = 1 / np.sqrt(m)
72         self.W = np.random.normal(0,1 / m**.5,[m,n]).
73         astype('float32')

```

```
70         self.B = np.zeros([n]).astype('float32')
71
72     def forward(self,X):
73         #TODO: fill in this function
74         self.input = X
75         out = np.dot(X, self.W) + self.B
76         if np.isnan(out).any():
77             print("NaN detected in Linear.forward")
78         return np.dot(X, self.W) + self.B
79
80     def backward(self,DY):
81         #TODO: fill in this function
82         DY = DY.reshape(-1, self.W.shape[1])
83         self.DW = np.dot(self.input.T, DY)
84         self.DB = np.sum(DY, axis=0)
85         return np.dot(DY, self.W.T)
86
87     def update(self,lr):
88         self.W -= lr*self.lr*self.DW
89         self.B -= lr*self.lr*self.DB
90
91     def average(self,nn,a):
92         self.W = a*nn.W + (1-a)*self.W
93         self.B = a*nn.B + (1-a)*self.B
94
95 class Tanh(Module):
96
97     def forward(self,X):
98         #TODO: fill in this function
99         self.input = X
100        return np.tanh(X)
101    def backward(self,DY):
102        #TODO: fill in this function
103        return DY * (1-np.tanh(self.input)**2)
104
105
```

```
1 import numpy as np
2
3 def rotate_3d_coordinates(coordinates, x_degrees,
4     y_degrees, z_degrees):
5     """
6     Rotates a set of 3D coordinates around the x, y,
7     and z axes by specified angles.
8
9     Args:
10     coordinates (np.array): An N x 3 array of 3D
11     coordinates.
12     x_degrees (float): The rotation angle around the
13     x-axis in degrees.
14     y_degrees (float): The rotation angle around the
15     y-axis in degrees.
16     z_degrees (float): The rotation angle around the
17     z-axis in degrees.
18
19     Returns:
20     np.array: The rotated coordinates: An N x 3 array
21     of 3D coordinates.
22     """
23     #TODO: write this function
24     Rx = np.array([[1, 0, 0],[0, np.cos(x_degrees), -
25     1 * np.sin(x_degrees)],[0, np.sin(x_degrees), np.cos(
26     x_degrees)]])
27     Ry = np.array([[np.cos(y_degrees), 0, np.sin(
28     y_degrees)],[0,1, 0],[-1* np.sin(y_degrees), 0, np.
29     cos(y_degrees)]])
30     Rz = np.array([[np.cos(z_degrees), -1 * np.sin(
31     z_degrees),0],[ np.sin(z_degrees), np.cos(z_degrees),
32     0],[0, 0, 1]])
33     rotated_coordinates = coordinates @ (Rx @ Ry @ Rz
34     ).T
35
36     return rotated_coordinates
37
```

```

1 import torch
2 import torch.nn.functional as nnF
3 import numpy as np
4
5 def burgers_pde_residual(x, t, u):
6     # x: (B, Nx, Nt)
7     # t: (B, Nx, Nt)
8     # u: (B, Nx, Nt)
9
10    #print(x.shape,t.shape)
11    nu = 0.01
12    dt = 1/(t.size()[2]-1)
13    dx = 1/(x.size()[1]-1)
14    # TODO
15    dudx = torch.zeros_like(u)
16    dudt = torch.zeros_like(u)
17    du2dx2 = torch.zeros_like(u)
18    zero_m = torch.zeros_like(u)
19
20    #print(dudx.shape)
21    '''
22    for k in range(0,x.size()[0]-1):
23        for i in range(1,t.size()[2]-1):
24            for j in range(1,x.size()[1]-1):
25
26                dudx[k][j][i] = (u[k, j + 1, i] - u[k
27, j - 1, i]) / (2*dx)
28                du2dx2[k][j][i] = (u[k, j + 1, i] + u
29[k, j - 1, i] - 2 * u[k, j, i]) / dx ** 2
30                dudt[k][j][i] = (u[k, j, i + 1] - u[k
31, j, i - 1]) / (2*dt)
32                #Edges using forward euler
33                dudx[k][0][i] = (u[k, 1, i] - u[k, 0
34, i]) / dx
35                dudx[k][-1][i] = (u[k, -1, i] - u[k
36, -2, i]) / dx
37                dudt[k][j][0] = (u[k, j, 1] - u[k, j
38, 0]) / dt

```

```

33         dudt[k][j][-1] = (u[k, j, - 1] - u[k
    , j, -2]) / dt
34         ##
35         dudx = (u[:, 2:u.size()[1]-1, :] - u[:, 0:u.size
    ()[1]-3, :]) / (2*dx)
36         du2dx2 = (u[:, 2:u.size()[1]-1, :] + u[:, 0:u.
    size()[1]-3, :] - 2 * u[:, 1:u.size()[1]-2, :]) / dx
    ** 2
37         dudt = (u[:, :, 2:u.size()[1]-1] - u[:, :, 0:u.size
    ()[1]-3]) / (2 * dt)'''
38
39         dudx[:, 1:x.shape[1]-2, :] = (u[:, 2:u.size()[1] -
    1, :] - u[:, 0:u.size()[1] - 3, :]) / (2 * dx)
40         du2dx2[:, 1:x.shape[1]-2, :] = (u[:, 2:u.size()[1]
    ] - 1, :] + u[:, 0:u.size()[1] - 3, :] - 2 * u[:, 1:u
    .size()[1] - 2, :]) / dx ** 2
41         dudt[:, :, 1:t.shape[2]-2] = (u[:, :, 2:u.size()[
    1] - 1] - u[:, :, 0:u.size()[1] - 3]) / (2 * dt)
42
43         dudx[:, 0, :] = (u[:, 1, :] - u[:, 0, :]) / (dx)
44         dudx[:, -1, :] = (u[:, -2, :] - u[:, -1, :]) / (
    dx)
45
46         dudt[:, :, 0] = (u[:, :, 1] - u[:, :, 0]) / ( dt)
47         dudt[:, :, -1] = (u[:, :, -2] - u[:, :, -1]) / (
    dt)
48
49         #du2dx2[:, 0, :] = (dudx[:, 1, :] - dudx[:, 0, :])/dx
50         #du2dx2[:, -1, :] = (dudx[:, -2, :] - dudx[:, -1
    , :]) / dx
51
52         #print(dudt.size(), dudx.size(), du2dx2.size())
53
54
55         resid = dudt +0.5* dudx**2 - nu * du2dx2
56         loss = torch.nn.MSELoss()
57         loss = loss(resid, zero_m)
58

```

```
59     bc_loss = torch.nn.MSELoss()
60     bc_loss = bc_loss(u[:,0,:],u[:, -1,:])
61
62     bc_loss2 = torch.nn.MSELoss()
63     bc_loss2 = bc_loss2(dudx[:,0,:],dudx[:, -1,:])
64
65     return loss+bc_loss+bc_loss2
66
67
68 def burgers_data_loss(predicted, target):
69     # Relative L2 Loss
70     # Predicted: (B, Nx, Nt)
71     # Target: (B, Nx, Nt)
72     # TODO
73     num = torch.norm(predicted - target, p =2)
74     den = torch.norm(target,p=2)
75     return num/den
76
77
```

```

1 import torch
2 from torch.utils.data import Dataset
3 import numpy as np
4 from utils import MatReader
5
6
7 class BurgersDataset(Dataset):
8     def __init__(self, data_path, train=True):
9         super().__init__()
10        self.train = train
11        dataloader = MatReader(data_path, to_cuda=
torch.cuda.is_available())
12        self.device = torch.device(
13            'cuda' if torch.cuda.is_available() else
'cpu')
14        self.ic_vals = dataloader.read_field("input"
) # Initial conditions
15        self.solutions = dataloader.read_field("
output") # Solution
16        self.tspan = dataloader.read_field("tspan")
17
18        nx = self.ic_vals.shape[1]
19        nt = self.tspan.shape[1]
20
21        # Discretized Grid (Nx, Nt, 2)
22        self.grid = torch.from_numpy(
23            np.mgrid[0: 1: 1 / nx, 0: 1: 1 / nt]).
permute(1, 2, 0).to(self.device)
24
25        # Quirk of data: Need to transpose x, t
26        self.solutions = self.solutions.permute(0, 2
, 1)
27
28    def __len__(self):
29        # TODO
30        return self.ic_vals.shape[0] # Number of
samples
31

```

```
32     def __getitem__(self, idx):
33         # TODO
34         ic = self.ic_vals[idx, :].to(self.device)
35         sol = self.solutions[idx, :, :].to(self.
device)
36         nx, nt, _ = self.grid.shape
37         ic_broadcast = ic.unsqueeze(1).expand(nx, nt
).unsqueeze(-1)
38         model_input = torch.cat([self.grid,
ic_broadcast], dim = -1)
39
40         return model_input, sol
41
42
```

```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5
6 class ConvNet2D(nn.Module):
7
8     # You may include additional arguments if you
wish.
9     def __init__(self):
10         # TODO
11         super().__init__()
12         self.conv1 = nn.Conv2d(in_channels=3,
13 out_channels=64, kernel_size=3, stride=1, padding=1)
14         self.Tanh1 = nn.Tanh()
15
16         self.conv2 = nn.Conv2d(64, 64, 3, 1, 1)
17         self.Tanh2 = nn.Tanh()
18
19         self.conv3 = nn.Conv2d(64, 64, 3, 1, 1)
20         self.Tanh3 = nn.Tanh()
21
22         self.conv4 = nn.Conv2d(64, 128, 1)
23         self.Tanh4 = nn.Tanh()
24         self.fc2 = nn.Conv2d(128, 1, kernel_size=1) #
Check during OH
25
26     def forward(self, x):
27         # TODO
28         x = x.permute(0, 3, 1, 2)
29         x = torch.tanh(self.conv1(x))
30         x = self.Tanh2(self.conv2(x))
31         x = self.Tanh3(self.conv3(x))
32         x = self.Tanh4(self.conv4(x))
33         x = self.fc2(x)
34         x = x.squeeze(1)
35         return x
```

```
1 import torch
2 import torch.nn.functional as nnF
3 import torch.optim as optim
4 from tqdm import tqdm
5 import matplotlib.pyplot as plt
6 import numpy as np
7
8 from data import BurgersDataset
9 from model import ConvNet2D
10 from pde import burgers_pde_residual,
    burgers_data_loss
11 from torch.utils.data import DataLoader
12
13 device = torch.device('cuda' if torch.cuda.
    is_available() else 'cpu')
14
15
16 def train():
17     burgers_train = BurgersDataset(
18         'data/Burgers_train_1000_visc_0.01.mat',
19         train=True)
20     burgers_validation = BurgersDataset(
21         'data/Burgers_test_50_visc_0.01.mat', train=
22         False)
23
24     # Hyperparameters
25     lr = 5e-3
26     batch_size = 16
27     epochs = 40
28
29     # Setup optimizer, model, data loader etc.
30     # TODO
31     train_loader = DataLoader(burgers_train,
32                               batch_size=batch_size, shuffle=True)
33     validation_loader = DataLoader(burgers_validation
34                                   , batch_size=batch_size, shuffle=False)
35
36     alpha = 0
```

```

33     model = ConvNet2D().to(device)
34     optimizer = optim.Adam(model.parameters(), lr=lr)
35     training_losses = []
36     pde_resid_loss = []
37     #loss_pde_only = burgers_pde_residual(inputs
    [::,::,0], inputs[:,::,1], target_val)
38     # Training Loop
39     for epoch in range(epochs):
40         model.train()
41         running_loss = 0.0
42         epoch_loss = 0.0
43         running_loss_pde = 0
44         for batch in train_loader:
45             inputs, targets = batch
46             inputs = inputs.float()
47             loss_pde_only = burgers_pde_residual(
    inputs[:, :, :, 0], inputs[:, :, :, 1], targets)
48             #print(inputs.shape)
49             print(loss_pde_only)
50             optimizer.zero_grad()
51             predicted = model(inputs)
52
53             #print(targets.shape)
54             loss1 = burgers_data_loss(predicted,
    targets)
55
56             loss2 = burgers_pde_residual(inputs
    [::,::,0], inputs[:,::,1], predicted)
57             loss = loss1 + alpha * loss2
58             loss.backward()
59             optimizer.step()
60             running_loss_pde += loss_pde_only
61             running_loss += loss.item()
62             epoch_loss += loss.item() * inputs.size(0
    )
63         epoch_loss /= len(train_loader.dataset)
64         training_losses.append(epoch_loss)
65         pde_resid_loss.append(running_loss_pde)

```

```
66
67     print(f"Epoch {epoch + 1}, Loss: {epoch_loss
        :.4f}")
68     np.save(f'training-both_losses.npy', np.array(
        training_losses))
69     np.save(f'pde_resid_loss.npy', np.array(
        pde_resid_loss))
70     torch.save(model, 'model_both_losses.pth')
71     # Validation Loop
72     # TODO
73     model.eval()
74     val_loss = 0.0
75     num_val_batches = 0
76     with torch.no_grad():
77         for x_val, target_val in validation_loader:
78             x_val, target_val = x_val.to(device),
            target_val.to(device)
79             x_val = x_val.float()
80             val_pred = model(x_val)
81             loss_val = burgers_data_loss(val_pred,
            target_val)
82             val_loss += loss_val.item()
83             num_val_batches += 1
84
85     print(f"Validation Loss: {val_loss /
        num_val_batches:.4f}")
86     np.save(f'training-rmses_both_losses.npy', np.
        array(val_loss / num_val_batches))
87     '''
88     plt.figure()
89     plt.plot(range(1, epochs), training_losses,
        label='Data Loss')
90     plt.title('Data Loss vs. Epoch')
91     plt.xlabel('Epochs')
92     plt.ylabel('Data Loss')
93     plt.legend()
94
95     plt.figure()
```

```
96     plt.contourf()
97     plt.plot(x_val[1,:,:], target_val[1,:,:], label
    = 'Data Loss')
98     plt.title('Data Loss vs. Epoch')
99     plt.xlabel('Epochs')
100    plt.ylabel('Data Loss')
101    plt.legend()
102    '''
103 if __name__ == '__main__':
104     torch.manual_seed(0)
105     train()
106
```

```
1 import torch
2 import scipy.io
3 import h5py
4 import numpy as np
5
6 class MatReader(object):
7     def __init__(self, file_path, to_torch=True,
8         to_cuda=False, to_float=True):
9         super(MatReader, self).__init__()
10
11         self.to_torch = to_torch
12         self.to_cuda = to_cuda
13         self.to_float = to_float
14
15         self.file_path = file_path
16
17         self.data = None
18         self.old_mat = None
19         self._load_file()
20
21     def _load_file(self):
22         try:
23             self.data = scipy.io.loadmat(self.
24 file_path)
25             self.old_mat = True
26         except:
27             self.data = h5py.File(self.file_path)
28             self.old_mat = False
29
30     def load_file(self, file_path):
31         self.file_path = file_path
32         self._load_file()
33
34     def read_field(self, field):
35         x = self.data[field]
36
37         if not self.old_mat:
38             x = x[()]
```

```
37         x = np.transpose(x, axes=range(len(x.  
    shape) - 1, -1, -1))  
38  
39         if self.to_float:  
40             x = x.astype(np.float32)  
41  
42         if self.to_torch:  
43             x = torch.from_numpy(x)  
44  
45             if self.to_cuda:  
46                 x = x.cuda()  
47  
48         return x  
49  
50     def set_cuda(self, to_cuda):  
51         self.to_cuda = to_cuda  
52  
53     def set_torch(self, to_torch):  
54         self.to_torch = to_torch  
55  
56     def set_float(self, to_float):  
57         self.to_float = to_float  
58
```

```
1 import torch
2 import torch.nn.functional as nnF
3 import torch.optim as optim
4 from tqdm import tqdm
5 import matplotlib.pyplot as plt
6 import numpy as np
7
8 from data import BurgersDataset
9 from model import ConvNet2D
10 from pde import burgers_pde_residual,
    burgers_data_loss
11 from torch.utils.data import DataLoader
12
13 device = torch.device('cuda' if torch.cuda.
    is_available() else 'cpu')
14
15
16 def pde_training():
17     burgers_train = BurgersDataset(
18         'data/Burgers_train_1000_visc_0.01.mat',
19         train=True)
20     burgers_validation = BurgersDataset(
21         'data/Burgers_test_50_visc_0.01.mat', train=
22         False)
23
24     # Hyperparameters
25     lr = 5e-3
26     batch_size = 16
27     epochs = 40
28
29     # Setup optimizer, model, data loader etc.
30     # TODO
31     train_loader = DataLoader(burgers_train,
32                               batch_size=batch_size, shuffle=True)
33     validation_loader = DataLoader(burgers_validation
34                                   , batch_size=batch_size, shuffle=False)
35
36     alpha = 0
```

```

33     model = ConvNet2D().to(device)
34     optimizer = optim.Adam(model.parameters(), lr=lr)
35     training_losses = []
36     pde_resid_loss = []
37     #loss_pde_only = burgers_pde_residual(inputs
    [::,::,0], inputs[:,::,1], target_val)
38     # Training Loop
39     for epoch in range(epochs):
40         model.train()
41         running_loss = 0.0
42         epoch_loss = 0.0
43         running_loss_pde = 0
44         for batch in train_loader:
45             inputs, targets = batch
46             inputs = inputs.float()
47             loss_pde_only = burgers_pde_residual(
    inputs[:, :, :, 0], inputs[:, :, :, 1], targets)
48             #print(inputs.shape)
49             print(loss_pde_only)
50
51             running_loss_pde += loss_pde_only
52
53             pde_resid_loss.append(running_loss_pde)
54
55             print(f"Epoch {epoch + 1}, Loss: {epoch_loss:
    .4f}")
56         np.save(f'pde_resid_loss.npy', np.array(
    pde_resid_loss))
57         # Validation Loop
58         # TODO
59
60 if __name__ == '__main__':
61     torch.manual_seed(0)
62     pde_training()
63

```

#1 (ii). See attached for plots of training & test losses (RMSEs & MAEs)

(iii). This NN doesn't learn anything about the molecules themselves beyond how a rigid structure affects atomization energy.

State-of-the-art ML models likely learn more about the molecules like the energy landscape from rotating, denoising or from data augmentations such as rotations. They also likely have more sophisticated architectures.

(iv). This form of a soft constraint allows the model to not place too much emphasis (i.e. overfit) on positions as they only partially affect E (only relative positions matter).

(v). Plots attached. Without data augmentations the validation RMSE was ~ 60 kcal/mol while w/ augmentation it was ~ 40 kcal/mol. This is bc the model was overfitting to absolute position. This is supported by the significantly lower training error of ~ 3 kcal/mol vs. ~ 30 kcal/mol.

(vi). Data augmentation is limited to ~~being the same data~~ to the angles at which we are limited by i.e. 360° in total. We also only rotate each datapoint 1 time.

(vii). We can use a graph representation which inherently ~~contains~~ is permutation invariant.

If the prediction task was a force vector rather than energy ($F = \nabla U$), only relative positions of atoms matter.

We can add another 'layer' to process the adjacency matrix into the correct dimensions for the input layer $(23, 23, 4) \rightarrow (92, 1)$.

(iii). Forces are equivariant to rotations while energies are invariant to rotations.

(embed as in nodes)

Input data $\checkmark$ as Bond angle & length would also help w/ graph representation

#2 (i). I notice that the general soln form looks correct, however, there are rough discontinuities present in the solution.

Switch

(iii) (b). The solutions are very smooth & vary a lot in a value

(iv) (c). Symbolic differentiation of the solution can be used (given we have labeled soln data)

(iv) (d). The loss is significant because it is not discretized very finely.

(v) (a). See attached. The pde residual loss is quite large relative to the data loss because it is not normalized. ($\alpha=1$, no regularization)

(v) (b). we can add a loss relative to the initial condition data loss

(vi) (a) the loss is much higher to start but decreases quickly when $\alpha=1$, the solutions are worse, likely due to the high PDE residual loss w/ the actual solution

(b). I think that ~~$\alpha=1$~~ $\alpha = \frac{1}{1500}$ is the best value because

it gives both the PDE residual & data loss similar magnitudes. It could be lowering the loss because the discretization is not very fine.

#2 (iv).

$$u(x+\Delta x) = u(x) + u'(x) \overset{(x+\Delta x - x)}{\Delta x} + O(\Delta x^2)$$

$$\Rightarrow \frac{u(x+\Delta x) - u(x)}{\Delta x} \approx u'(x)$$

$$u(x-\Delta x) = u(x) + u'(x)(x-\Delta x - x) + O(\Delta x^2)$$

$$\rightarrow \frac{u(x) - u(x-\Delta x)}{\Delta x} \approx u'(x)$$

take avg.

$$u'(x) = \frac{u(x+\Delta x) - u(x-\Delta x)}{2\Delta x}$$

2nd derivative:

$$u(x+\Delta x) = u(x) + u'(x)(\Delta x) + \frac{u''}{2} \Delta x^2$$

$$u(x-\Delta x) = u(x) - u'(x)\Delta x + \frac{u''}{2} \Delta x^2$$

$$+ \frac{u(x+\Delta x) - 2u(x) + u(x-\Delta x)}{\Delta x^2} = u''(x) \quad \text{\& simplify}$$

$$\frac{2}{2\Delta x} \left(u'(x) = \frac{u(x+\Delta x) - u(x-\Delta x)}{2\Delta x} \right)$$

$$u''(x) = \frac{u'(x+\Delta x) - u'(x-\Delta x)}{2\Delta x}$$

$$u'(x+\Delta x) = \frac{u(x+2\Delta x) - u(x)}{2\Delta x}$$

$$u'(x-\Delta x) = \frac{u(x) - u(x-2\Delta x)}{2\Delta x}$$

$2\Delta x \rightarrow \Delta x$;

$$u''(x) = \frac{u(x+\Delta x) - u(x)}{\Delta x} - \frac{u(x) - u(x-\Delta x)}{\Delta x}$$

However, simpler method

demonstrates

that error scales w/ Δx^2

(Since all terms scale w/ Δx^2)

#1 (ii). See attached for plots of training & test losses (RMSEs & MAEs)

(iii). This NN doesn't learn anything about the molecules themselves beyond how a rigid structure affects atomization energy.

State-of-the-art ML models likely learn more about the molecules like the energy landscape from rotating, denoising or from data augmentations such as rotations. They also likely have more sophisticated architectures.

(iv). This form of a soft constraint allows the model to not place too much emphasis (i.e. overfit) on positions as they only partially affect E (only relative positions matter).

(v). Plots attached. Without data augmentations the validation RMSE was ~ 60 kcal/mol while w/ augmentation it was ~ 40 kcal/mol. This is bc the model was overfitting to absolute position. This is supported by the significantly lower training error of ~ 3 kcal/mol vs. ~ 30 kcal/mol.

(vi). Data augmentation is limited to ~~being the same data~~ to the angles at which we are limited by i.e. 360° in total. We also only rotate each datapoint 1 time.

(vii). We can use a graph representation which inherently ~~contains~~ is permutation invariant.

If the prediction task was a force vector rather than energy ($F = \nabla U$), only relative positions of atoms matter.

We can add another 'layer' to process the adjacency matrix into the correct dimensions for the input layer $(23, 23, 4) \rightarrow (92, 1)$.

(iii). Forces are equivariant to rotations while energies are invariant to rotations.

(embed as in nodes)

Input data $\checkmark$ as Bond angle & length would also help w/ graph representation

#2 (i). I notice that the general soln form looks correct, however, there are rough discontinuities present in the solution.

Switch

(iii) (b). The solutions are very smooth & vary a lot in a value

(iv) (c). Symbolic differentiation of the solution can be used (given we have labeled soln data)

(iv) (d). The loss is significant because it is not discretized very finely.

(v) (a). See attached. The pde residual loss is quite large relative to the data loss because it is not normalized. ($\alpha=1$, no regularization)

(v) (b). we can add a loss relative to the initial condition data loss

(vi) (a) the loss is much higher to start but decreases quickly when $\alpha=1$, the solutions are worse, likely due to the high PDE residual loss w/ the actual solution

(b). I think that ~~$\alpha=1$~~ $\alpha = \frac{1}{1500}$ is the best value because

it gives both the PDE residual & data loss similar magnitudes. It could be lowering the loss because the discretization is not very fine.

#2 (iv).

$$u(x+\Delta x) = u(x) + u'(x) \overset{(x+\Delta x - x)}{\Delta x} + O(\Delta x^2)$$

$$\Rightarrow \frac{u(x+\Delta x) - u(x)}{\Delta x} \hat{=} u'(x)$$

$$u(x-\Delta x) = u(x) + u'(x)(x-\Delta x - x) + O(\Delta x^2)$$

$$\rightarrow \frac{u(x) - u(x-\Delta x)}{\Delta x} \hat{=} u'(x)$$

take avg.

$$u'(x) = \frac{u(x+\Delta x) - u(x-\Delta x)}{2\Delta x}$$

2nd derivative:

$$u(x+\Delta x) = u(x) + u'(x)(\Delta x) + \frac{u''}{2} \Delta x^2$$

$$u(x-\Delta x) = u(x) - u'(x)\Delta x + \frac{u''}{2} \Delta x^2$$

$$+ \frac{u(x+\Delta x) - 2u(x) + u(x-\Delta x)}{\Delta x^2} = u''(x) \quad \& \text{ simplify}$$

$$\frac{2}{2\Delta x} \left(u'(x) = \frac{u(x+\Delta x) - u(x-\Delta x)}{2\Delta x} \right)$$

$$u''(x) = \frac{u'(x+\Delta x) - u'(x-\Delta x)}{2\Delta x}$$

$$u'(x+\Delta x) = \frac{u(x+2\Delta x) - u(x)}{2\Delta x}$$

$$u'(x-\Delta x) = \frac{u(x) - u(x-2\Delta x)}{2\Delta x}$$

$2\Delta x \rightarrow \Delta x;$

$$u''(x) = \frac{u(x+\Delta x) - u(x)}{\Delta x} - \frac{u(x) - u(x-\Delta x)}{\Delta x}$$

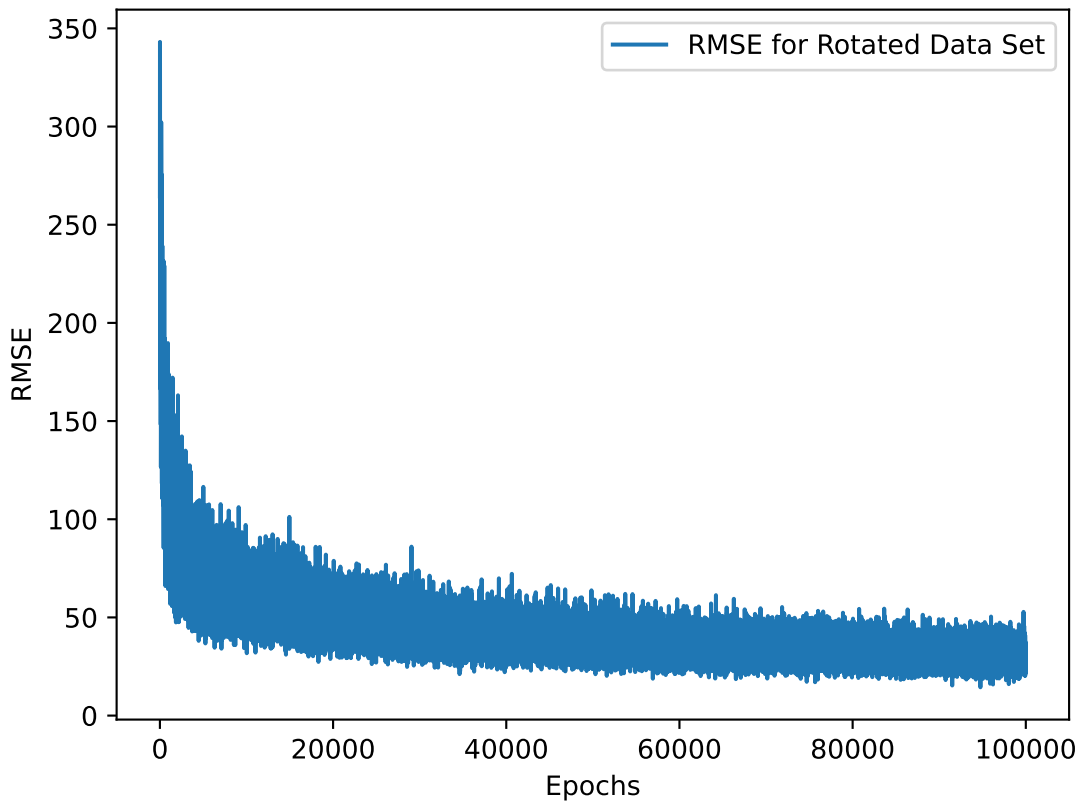
However, simpler method

demonstrates

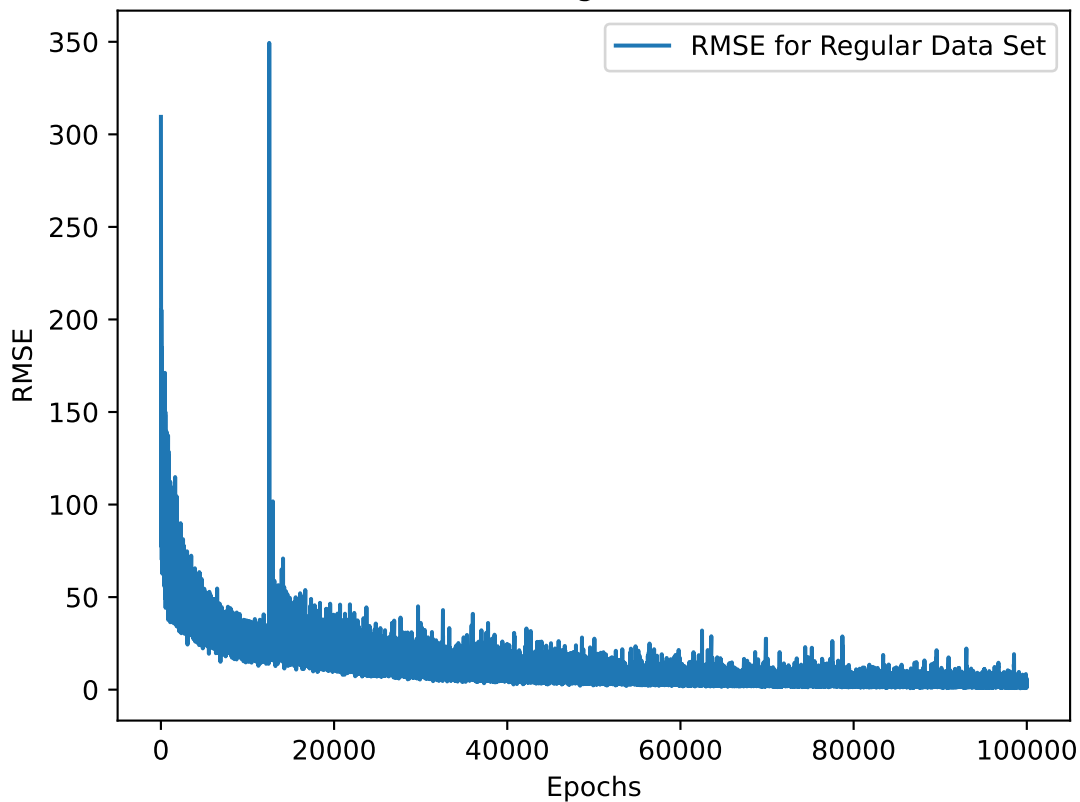
that error scales w/ Δx^2

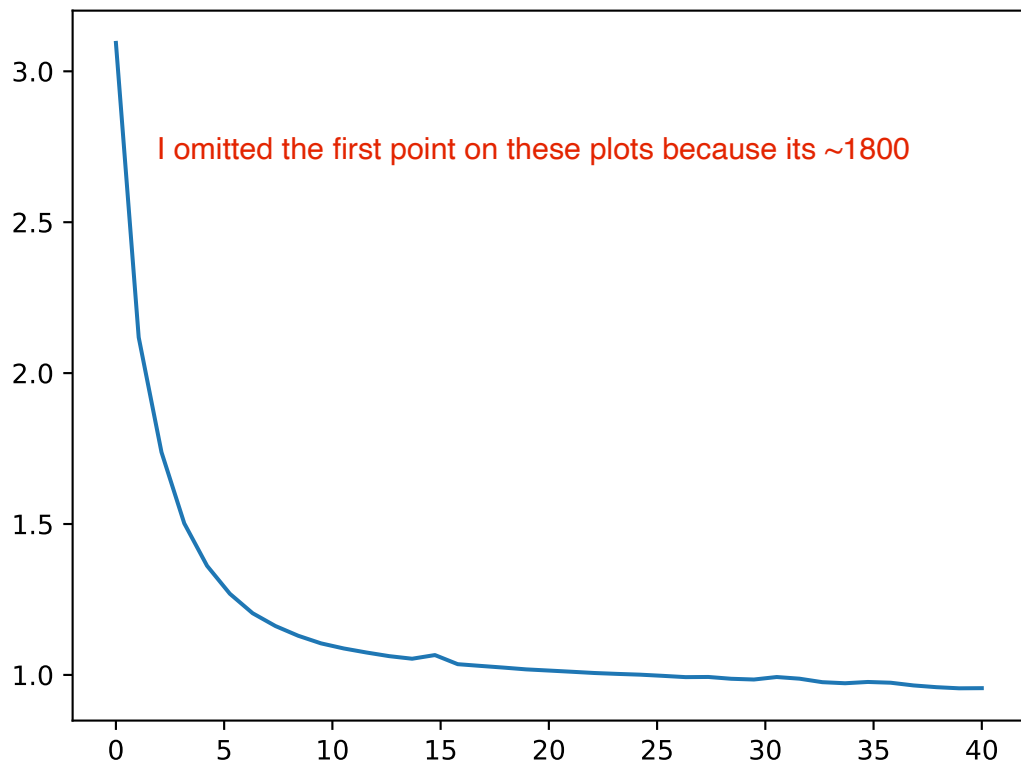
(Since all terms scale w/ Δx^2)

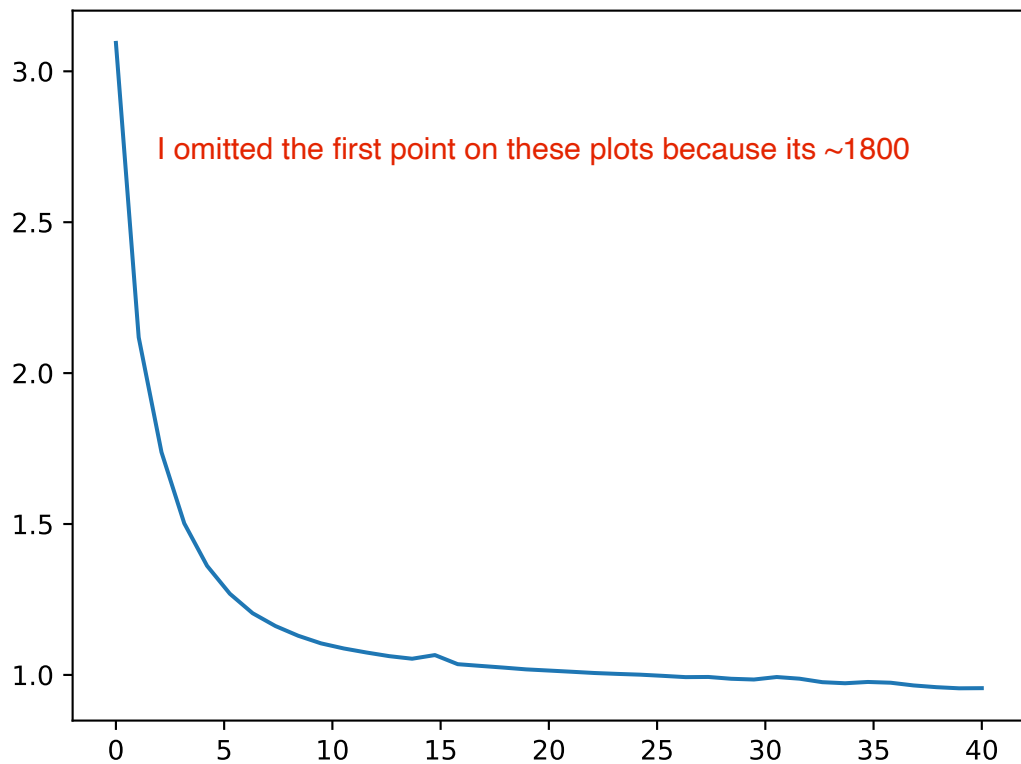
RMSE for Rotated Data Set

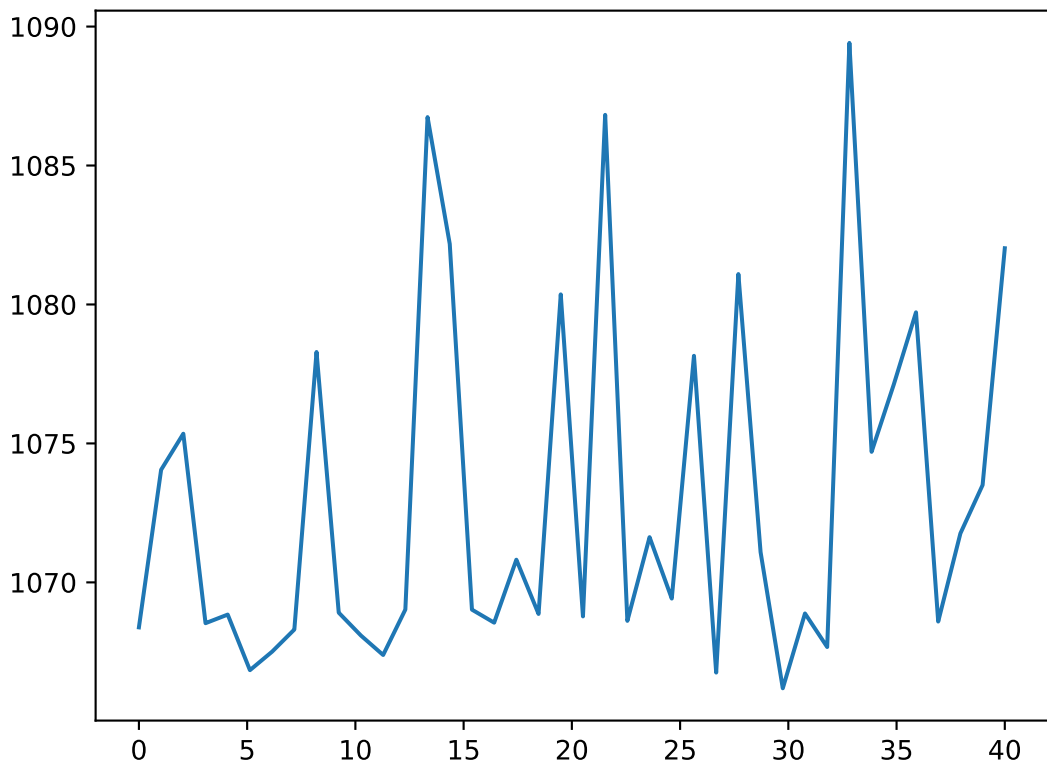


RMSE for Regular Data Set

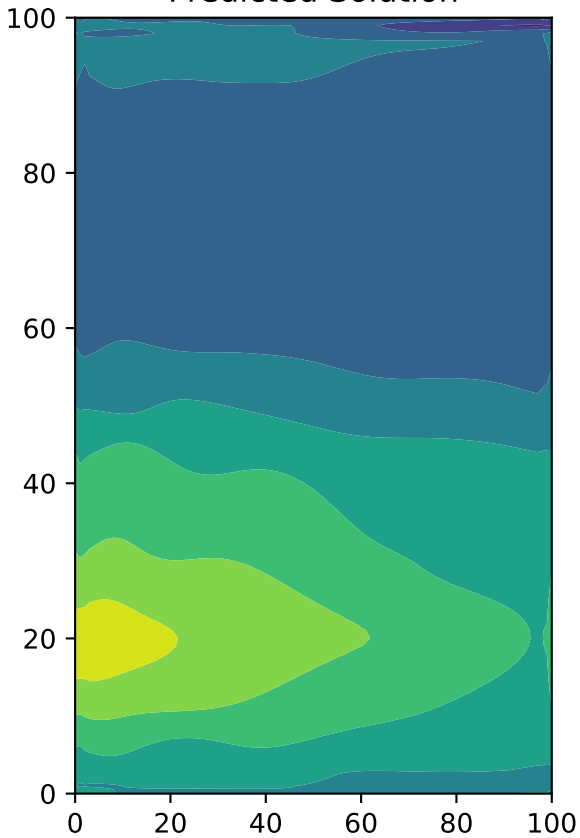




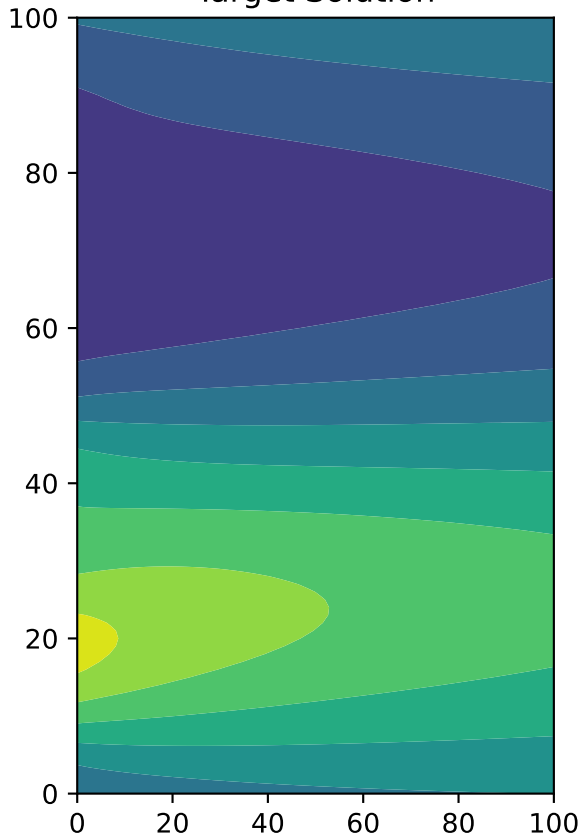




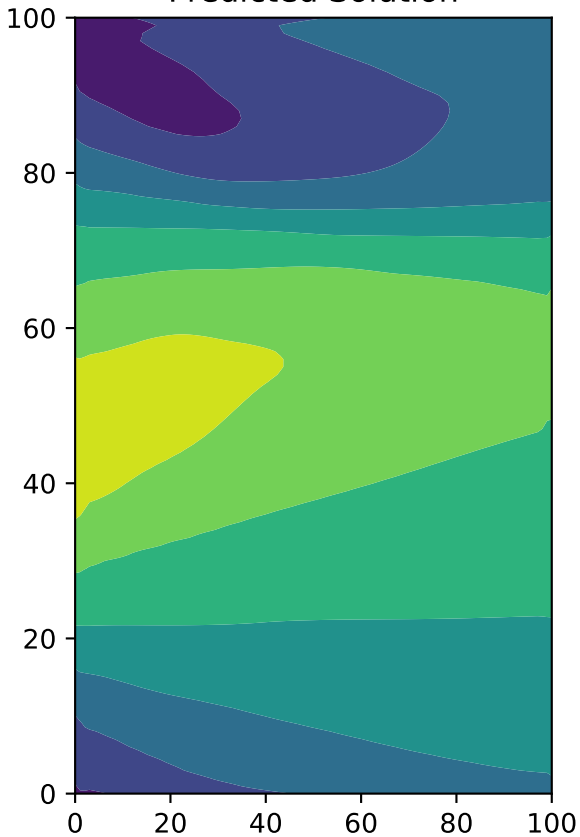
Predicted Solution



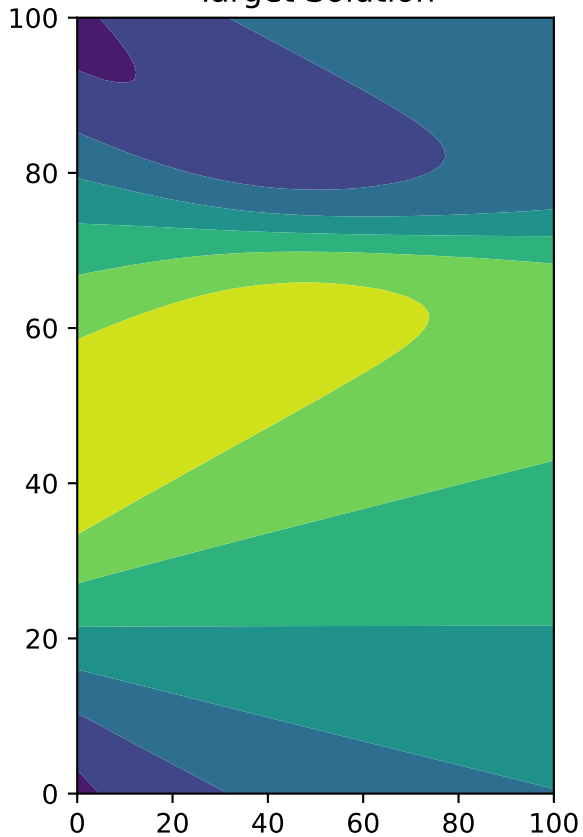
Target Solution



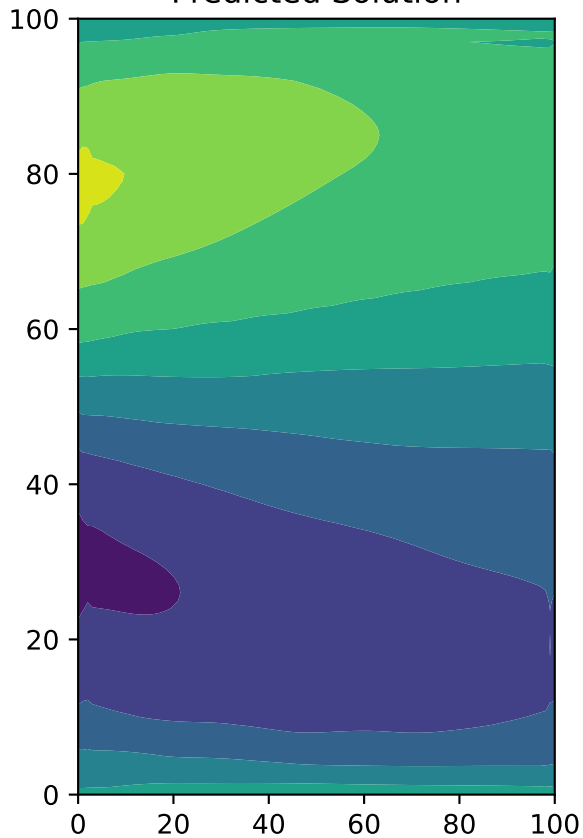
Predicted Solution



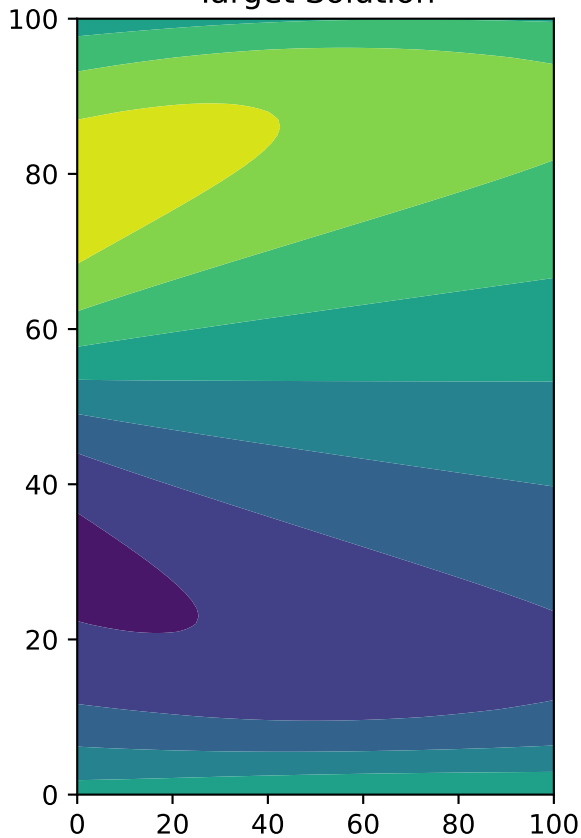
Target Solution



Predicted Solution



Target Solution



results of nn-augment=True.pkl after 99900 iterations
training set

MAE: 24.25 kcal/mol

RMSE: 30.55 kcal/mol

test set

MAE: 26.68 kcal/mol

RMSE: 37.61 kcal/mol

(.venv) alankstoev@wifi-10-40-80-15 problem1_qm7 %

```
results of nn-augment=False.pkl after 99900 iterations
training set
MAE:  1.15 kcal/mol
RMSE: 2.35 kcal/mol
test set
MAE:  42.93 kcal/mol
RMSE: 58.32 kcal/mol
(.venv) alankstoev@wifi-10-40-80-15 problem1_qm7 %
```

(2) This is PDE vs actual solution loss, Mean and Std is per batch

Mean of: 1072.9489

St. Dev of: 6.2501793

PDE only Validation loss:
1.1074

Data only validation loss: 0.25

results of nn-augment=False.pkl after 99900 iterations
training set

MAE: 1.15 kcal/mol

RMSE: 2.35 kcal/mol

test set

MAE: 42.93 kcal/mol

RMSE: 58.32 kcal/mol

(.venv) alankstoev@wifi-10-40-80-15 problem1_qm7 %

results of nn-augment=True.pkl after 99900 iterations
training set

MAE: 24.25 kcal/mol

RMSE: 30.55 kcal/mol

test set

MAE: 26.68 kcal/mol

RMSE: 37.61 kcal/mol

(.venv) alankstoev@wifi-10-40-80-15 problem1_qm7 %

Mean of: 1072.9489

St. Dev of: 6.2501793

results of nn-augment=False.pkl after 99900 iterations
training set

MAE: 1.15 kcal/mol

RMSE: 2.35 kcal/mol

test set

MAE: 42.93 kcal/mol

RMSE: 58.32 kcal/mol

(.venv) alankstoev@wifi-10-40-80-15 problem1_qm7 %

results of nn-augment=True.pkl after 99900 iterations
training set

MAE: 24.25 kcal/mol

RMSE: 30.55 kcal/mol

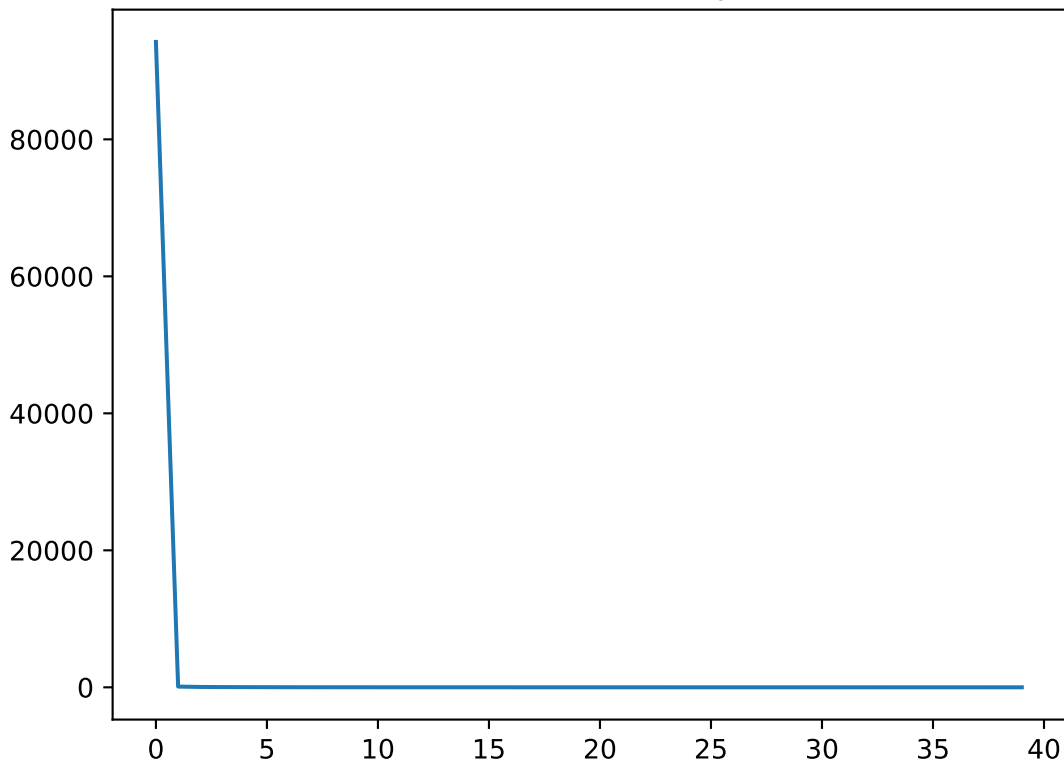
test set

MAE: 26.68 kcal/mol

RMSE: 37.61 kcal/mol

(.venv) alankstoev@wifi-10-40-80-15 problem1_qm7 %

PDE Residual Loss for $\alpha = 1$



Data Loss for $\alpha = 1$

